

Magíster en Ciencia de Datos  
para la Innovación

# FUNDAMENTOS DE BASES DE DATOS Y ALGORITMOS: Parte 1

**JOSÉ FUENTES**

Ingeniero Civil Informático

Doctor en Ciencias de la Computación

[jfuentess@inf.udec.cl](mailto:jfuentess@inf.udec.cl)

CLASE #4

## Funciones



Facultad de Ingeniería  
Universidad de Concepción



CENTRO DE DATOS E  
INTELIGENCIA ARTIFICIAL  
UNIVERSIDAD DE CONCEPCIÓN

# Funciones en Python

## Notación Python

```
L = [4, 8, 0, 5, -1, 7]
M = [20, 0, 9, 11, 15, 21]

max_L = L[0]
for e in L:
    if max_L < e:
        max_L = e
print('Máximo de L: ', max_L)

max_M = M[0]
for e in M:
    if max_M < e:
        max_M = e
print('Máximo de M: ', max_M)
```

**VS**

## Notación Python

```
def maximo(A):
    max_A = L[0]
    for e in A:
        if max_A < e:
            max_A = e

    return max_A

L = [4, 8, 0, 5, -1, 7]
M = [20, 0, 9, 11, 15, 21]

print('Máximo de L: ', maximo(L))
print('Máximo de M: ', maximo(M))
```

# Funciones en Python

```
def nombre_función(parámetro 1, ..., parámetro n):  
    ... Código de la función ...  
    return variable_a_retornar
```

## Algunas funciones que ya hemos visto

```
print('Valor ', x)
```

```
type(x)
```

```
min(x)
```

```
max(x)
```

```
range(0, n)
```

```
len(L)
```

```
int(x)
```

```
input()
```

# Funciones en Python: ejemplo

```
# ¿Qué hace esta función?  
def funcion1(valor1, valor2):  
    c = 0  
    r = 0  
    while valor1 >= valor2:  
        c = c + 1  
        valor1 = valor1 - valor2  
  
    r = valor1  
  
    return c, r
```

```
a = 14  
b = 3  
x, y = funcion1(a, b)
```

**Seguimiento:**

# Funciones en Python: Ámbito o scope de variables

```
def funcion2(a):  
    a = 2*a  
    b = 10  
    return a  
  
b = 5  
c = 50  
d = funcion2(c)  
  
print('a: ', a) # Genera un error (!)  
print('b: ', b)  
print('c: ', c)  
print('d: ', d)
```

¿Qué ocurre?

# Funciones en Python: llamadas de funciones

**Seguimiento:**

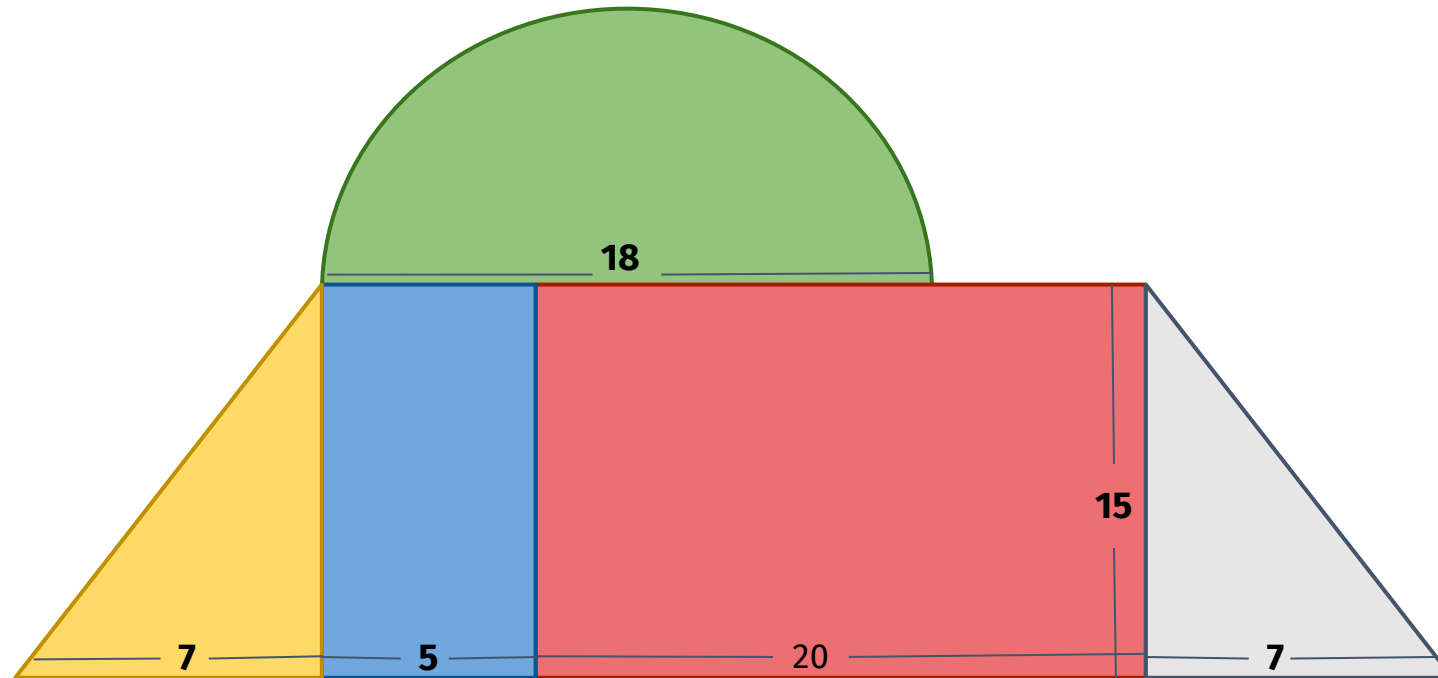
```
def min_pos(L, a, b):  
    pos = a  
    for i in range(a+1,b):  
        if L[i] < L[pos]:  
            pos = i  
  
    return pos  
  
def ordenar(L):  
    n = len(L)  
    for i in range(0, n):  
        pos = min_pos(L, i, n)  
        # Intercambiamos los elementos  
        # en las posiciones i y pos  
        L[i], L[pos] = L[pos], L[i]  
  
L = [1, 0, 9, -5]  
ordenar(L)  
print(L)
```

# Funciones en Python: funciones recursivas

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n*factorial(n-1)  
  
print(factorial(10))
```

**Seguimiento:**

# Ejemplo: áreas de polígonos



¡A PRACTICAR!



```
poligonos = [('triangulo', 7, 15), ('rectangulo', 5, 15), ('rectangulo', 20, 15),  
             ('triangulo', 7, 15), ('semicirc', 18)]
```